

1. from scratch
2. reduction

Definition

Let A and B be two languages over Σ_A and Σ_B , respectively.

A reduction from A to B is a computable function $f: \Sigma_A^* \rightarrow \Sigma_B^*$
 (recursive)

$\downarrow$ $\downarrow$ $\downarrow$
 pass " 队友
 the exam 金 金
 $\bar{w}$ w $f(w)$

such that
 $w \in A$ if and only if $f(w) \in B$.

Theorem

Suppose that there is a reduction f from A to B .

If B is recursive, so is A . ($A \leq B$)

Proof:

M_B decides B

M_A = on input w

1. compute $f(w)$

2. run M_B on $f(w)$

3. If M_B accepts $f(w)$.

4. accept w

5. else

 reject w

Corollary

If A is not recursive,

then B is not recursive.

Theorem

The following problems are undecidable.

(a) $A = \{ \langle M \rangle : M \text{ is a TM that halts on } e \}$

we show a reduction from H to A



$\{ \langle M \rangle \langle w \rangle : M \text{ is a TM that halts on } w \}$

$f: \text{inputs of H} \rightarrow \text{inputs of A}$

(invalid input $\rightarrow$ invalid input of A trivial
 $\langle M \rangle \langle w \rangle$ $\rightarrow$ $\langle M \rangle$?)

Given a TM M and a string w ,

M_w = on input w

1. run M on w

M_w halts on some input



$\langle M \rangle \langle w \rangle \in B$

$\Leftrightarrow$ M_w halts on e

$\langle M \rangle \in A$

M_w halts on every input



M halts on w

$\langle M \rangle \langle w \rangle \in H$

(b) $B = \{ \langle M \rangle : M \text{ is a TM that halts on some string} \}$

$H \leq B$

$f(\langle M \rangle \langle w \rangle) = \langle M \rangle$

(c) $C = \{ "M" : M \text{ is a TM that halts on every input} \}$

$$H \leq C$$

$$f("M" "w") = "Mw"$$

(d) $D = \{ "M_1" "M_2" : M_1 \text{ and } M_2 \text{ are two TMs that halt on the same input string} \}$

$$C \leq D$$

Let M^* be a TM with $s \in H \Rightarrow M^*$ halts on every input.

$$\underline{f("M")} = "M" "M^"$$

$$\underline{"M"} \in C \text{ iff } \underline{"M" "M^"} \in D$$

(e) $\bar{E}_1 = \{ "M" : M \text{ is a TM with } L(M) \text{ is regular} \}$

$\bar{E}_2 = \{ "$ " is context-free

$\bar{E}_3 = \{ "$ " is recursive?

$$H \leq \bar{E}_1, \bar{E}_2, \bar{E}_3$$

$$f("M" "w") = \underline{"Mw"}$$

Given M and w ,

define $Mw =$ on input v

1. run M on w

2. run U on v

regular, context-free
recursive

universal
TM

$$L(U) = H$$

$$L(M_w) = \begin{cases} \phi & \text{if } "M" "w" \notin H \\ L(U) = H & \text{if } "M" "w" \in H \end{cases}$$

not recursive
nor context-free
nor regular.

$L(M_w)$ is regular / context-free / recursive. $("M_w" \in \bar{E}_1, \bar{E}_2, \bar{E}_3)$
 $("M_w" \notin \bar{E}_1, \bar{E}_2, \bar{E}_3)$
 $"M" "w" \notin H$

†) There is a certain fixed Turing Machine M , for which the following problem is undecidable,

Given w , does M halt on w ?

Halting problem: $\frac{"M"}{\uparrow}$ $\frac{"w"}{\uparrow}$
 fixed input

is $w \in L(M)$?

$\downarrow$
 $L(M)$

$$M = U$$

M^* with $S \in H$

$L(U) = H \leftarrow$ not recursive

Rice theorem

Suppose that L is a proper, non-empty subset of all recursively enumerable language. The following is undecidable.

Given a TM M , is $L(M) \in L$?

$S = \{ \langle M \rangle : M \text{ is a TM with } L(M) \in L \}$

Proof:

Assume $\emptyset \notin L$. (If $\emptyset \in L$, $L = \bar{L}$)

Let A be a language in L . Let $\underline{M_A}$ be a TM that semidecides A .

$H \leq S$

Given M and w , define a TM $f(\langle M \rangle \langle w \rangle)$ as follow.

$f(\langle M \rangle \langle w \rangle) =$ on input u

1. run M on w

2. run $\underline{M_A}$ on u

$L(f(\langle M \rangle \langle w \rangle)) \begin{cases} \emptyset \notin L & \text{if } \langle M \rangle \langle w \rangle \notin H \\ L(M_A) = A \in L & \text{if } \langle M \rangle \langle w \rangle \in H \end{cases}$

$\langle M \rangle \langle w \rangle \in H \iff f(\langle M \rangle \langle w \rangle) \in S$

(a) Given a TM M , does M halt on e ?

is $e \in L(M)$?

↓

is $L(M) \in \mathcal{L}$ where $\mathcal{L} = \{ L : L \text{ is R.E. and } e \in L \}$?

(b) Given a TM M , does M halt on some input?

↓

is $L(M) \neq \emptyset$?

↓

is $L(M) \in \mathcal{L}$ where $\mathcal{L} = \{ L : L \text{ is R.E. and } L \neq \emptyset \}$?

(c) Given a TM M , does M halt on every input?

↓

is $L(M) = \Sigma^*$?

↓

$\mathcal{L} = \{ \Sigma^* \}$

(e) is $L(M)$ regular $\neq$ context-free $\neq$ recursive?

↓

↓

↓

$\mathcal{L} =$

1. enumerator

2. unrestricted grammar

3. numerical function

> TM

Enumerator

Definition

We say that a TM M enumerates a language L if for some state q ,

$$L = \{ w : (s, \triangleright \sqcup) \vdash_m^* (q, \triangleright \sqcup w) \}$$

↓

Turing-enumerable

recursively-enumerable

↓
semidecide.

Theorem

A language L is recursively-enumerable iff it is Turing-enumerable.

Proof:

Assume that L is infinite.

⇐) Let M be a TM that enumerates L .

Construct a TM M' to semidecide L .

M' = on input w

1. run M to enumerate L
2. Every time M outputs a string, compare it with w , and halt if they match

⇒) Let M be a TM that semidecides L .

Construct M' to enumerate L .

Name all strings over Σ as $s_1, s_2, \dots$

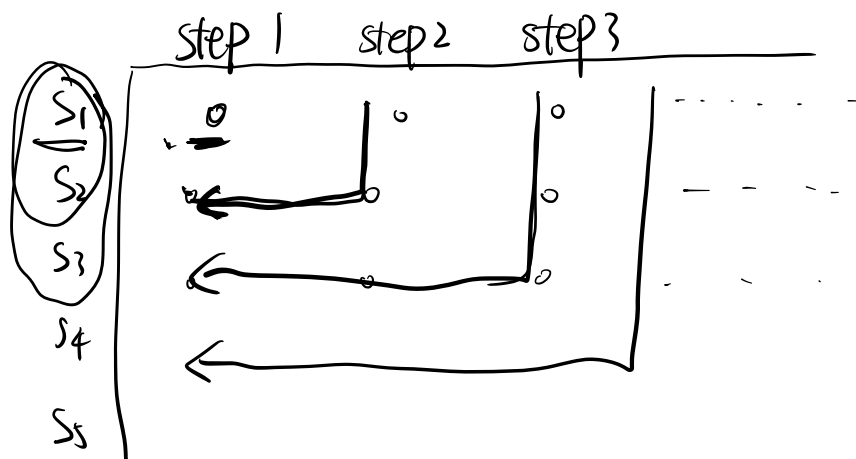
in lexicographical order

short < long

$\Sigma = \{0, 1\}$, $\epsilon, 0, 1, 00, 01, 10, 11, \dots$

$M' =$

1. for $i = 1, 2, 3, \dots$
2. run M on s_i (to see if $s_i \in L$)
3. if M halts on s_i
4. output s_i



$M' =$

1. for $i = 1, 2, 3, \dots$
2. generate $s_1, s_2, \dots, s_i$
3. run M on each of $s_1, \dots, s_i$ at most i steps
4. M halts on s_j within i steps
output s_j

lexicographically Turing-enumerable

Definition

Let M be a TM that enumerates L .

We say M lexicographically enumerate L if whenever

$(q, \sqcup w) \vdash_M^+ (q, \sqcup w')$ (w' is output after w)
 w' is after w in lexico. order.
 output state at least one step

Theorem

A language is recursive iff it is lexico. Turing - enumerable.

Proof:

$\Rightarrow$) Let M a TM that decide L .

M' to lexico. enumerate L .

$M' =$

1. generate all strings over Σ in lexico. order
2. Every time it generates a string, w ,
3. run M on w (see if $w \in L$)
4. output w if M accept w .

$\Leftarrow$) Let M lexico. enumerate L
 M' decides L .

$M' =$ on input w

1. run M to enumerate L until M outputs a string is after w in lexico. order.
2. If w is among the output strings.
 accept w

3. else
reject w .